

企业级应用软件设计与开发大作业报告

封面页（必填）

字段	内容
课程名称	企业级应用软件设计与开发
项目名称	Agentic RAG 知识库检索服务
方向	方向一：Agentic AI 原生开发
学号	2025302942
姓名	付洋杨
专业	计算机技术 / 软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

摘要

本项目实现了一个基于 LangGraph 的 Agentic RAG 知识库检索服务。传统 RAG 常采用固定的“向量召回 → 拼接上下文 → LLM 回答”流程，面对专有名词、技术型号、API 名称、长文本上下文补全等场景时，容易出现召回路径单一、关键词不敏感、上下文不足的问题。本系统将 LLM 设计为检索 Agent，由 Agent 根据问题类型自主选择向量检索、BM25 检索或双路混合检索，并在每轮检索后评估资料是否充分，不充分时携带缺失信息继续检索。

系统支持离线索引构建、父子块切分、BGE-M3 embedding 缓存、ChromaDB 向量持久化、BM25 语料持久化、RRF 融合、BGE Reranker 精排和 parent document 回填。测试部分覆盖索引写入完整性、RAG 排序指标、索引一致性和真实落盘索引检查。当前落盘索引中 Chroma 与 BM25 均为 12341 个子块，parent_store 为 8603 条，真实一致性检查通过。

一、选题背景与设计思想（20 分）

本章从问题价值、现有方案不足、项目创新点和技术路线四个角度说明选题依据。

1.1 问题定义

企业级知识库通常包含产品文档、技术规范、接口说明、运维记录和历史问答。用户提问既可能是“迁移学习是什么”这类语义宽泛问题，也可能是“React 18 Suspense API”这类强关键词问题，还可能是“对比微服务和单体架构在可观测性上的差异”这类需要多路召回和综合判断的问题。单一路径的向量检索无法稳定覆盖这些查询类型。

本项目要解决的问题是：让检索系统具备查询理解、工具选择、结果评估和上下文补全能力，使 RAG 从固定流水线升级为可自适应的 Agentic 检索流程。

从工程角度看，RAG 的核心瓶颈并不只是“有没有向量库”，而是“在不同问题下如何选择合适的证据”。向量检索擅长处理语义相似和同义改写，但对于版本号、函数名、论文缩写、错误码等离散 token，向量距离可能被周围语义平均掉；BM25 对关键词敏感，但遇到同义表达、长问题拆解和跨语言描述时又容易漏召回。因此系统需要把查询理解、检索工具选择和结果充分性判断放到同一个闭环里，而不是把所有问题都交给单一检索器处理。

1.2 现有方案不足

方案	优点	不足
纯向量检索 RAG	对同义表达、概念解释友好	对 API 名称、型号、版本号等精准词不稳定
纯 BM25 检索	精确匹配能力强，易解释	对语义改写、模糊表达和跨语言表达不友好
一次性混合检索	能兼顾两类召回	缺少“结果是否足够”的判断，无法主动重试
Agentic RAG	可按问题类型选择工具并迭代	工程复杂度更高，需要状态管理、工具约束和测试保障

本项目采用 Agentic RAG，是因为课程方向强调 Agentic AI 原生开发；同时该方案能体现企业级软件中“可扩展、可测试、可观测”的系统设计思想。

1.3 项目价值

项目价值主要体现在三点。第一，检索路径由 Agent 动态决策，能根据查询类型选择 BM25、向量检索或双路融合。第二，索引阶段同时维护 ChromaDB、BM25 语料和 parent_store，避免在线阶段只返回碎片化子块。第三，测试脚本把索引一致性和排序指标公式纳入自动化检查，降低后续扩展时的数据损坏风险。

创新点并不是简单把 LLM 接到 RAG 前面，而是让 LLM 承担“检索规划器”和“证据审查器”两类角色。规划器负责判断问题属于精准查询、模糊查询还是复杂查询；证据审查器负责判断当前累计文档是否足够回答原问题。如果不足，系统会把缺失信息作为 missing_hint 输入下一轮检索，使检索过程具有反馈调节能力。

1.4 技术路线

技术路线为“离线构建多路索引 + 在线 Agentic 检索”。离线阶段加载多格式数据，进行父块与子块切分，子块用于召回，父块用于最终回答上下文；在线阶段由 LangGraph 编排 Agent 工作流，LLM 先改写查询，再调用工具，最后根据充分性评估决定结束或继续检索。

该路线把高成本、可批处理的工作放在离线阶段完成，例如 embedding 计算、ChromaDB 写入、BM25 语料构建和 parent_store 保存；在线阶段只做轻量状态流转和检索决策，降低查询延迟。这样既能保留 Agentic RAG 的灵活性，又不会让每次查询都重新处理原始文档。

二、Specs 规格文档（20 分）

本章给出 Product Spec、Architecture Spec、API Spec 和 SDD 约束，使需求、架构和实现之间保持可追踪关系。

2.1 Product Spec

项目	规格
目标用户	需要在本地或企业内网构建知识库问答能力的开发者、课程项目评审者、技术文档维护者
核心目标	构建可解释、可扩展、可测试的 Agentic RAG 检索服务

项目	规格
主要输入	用户自然语言问题、对话历史、本地文档或 HuggingFace 数据集
主要输出	final_docs：经父块回填后的高相关上下文文档
核心能力	查询改写、工具路由、向量检索、BM25 检索、RRF 融合、充分性评估、父块回填
非功能需求	本地持久化、增量构建、缓存复用、索引一致性检查、环境变量管理 API Key
成功标准	能构建索引、执行查询、通过自动化测试，且检索流程可通过文档和图示解释

2.2 Architecture Spec

模块	文件	职责
CLI 入口	src/main.py	提供 build、query、serve 子命令
胶水层	src/cli.py	加载配置、构建索引、组装 GraphRetriever
索引构建	src/indexer.py	文档加载、分块、向量写入、BM25/parent_store 持久化
Agentic 检索图	src/graph_retriever.py	LangGraph 状态机、工具调用、RRF、充分性评估
工具定义	src/tools.py	将 vector_search 与 bm25_search 暴露为 Agent 可调用工具
数据加载	src/multiloader.py	支持 JSON、JSONL、Markdown、CSV、PDF、HTML 和 HuggingFace 数据
文本切分	src/hybridtextsplitter.py	父子块切分、轻量去重、自适应 overlap
Embedding 缓存	src/cachembedding.py	基于 SHA256 的 embedding 磁盘缓存
检索组件	src/retriever/	向量检索、BM25 检索、CrossEncoder 精排

2.3 API Spec

命令行 API

命令	参数	说明
python src/main.py build	--sample N、--datasets a,b、--mode offline/online	构建或追加本地索引
python src/main.py query "问题"	--topk N	执行一次 Agentic 检索并打印文档

命令	参数	说明
<code>python src/main.py</code> <code>serve</code>	无	预留 LangGraph API 服务入口

Python 内部接口

接口	输入	输出
<code>IndexBuilder.build()</code>	数据路径、DB 路径、缓存路径	Chroma 实例
<code>GraphRetriever.invoke(query, history, thread_id)</code>	查询、历史、会话 ID	<code>list[dict]</code> <code>final_docs</code>
<code>BM25Retriever.invoke(query, k)</code>	查询和 top-k	<code>list[Document]</code>
<code>ReRanker.rerank(query, docs, top_k)</code>	查询、候选文档	精排后的文档

2.4 SDD 核心设计约束

约束	设计
工具调用可控	System Prompt 明确三类查询的路由规则
状态可追踪	<code>RetrievalState</code> 保存 query、messages、工具结果、累积文档、充分性结果
数据一致	ChromaDB、BM25 语料和 parent_store 共用 chunk hash / parent_id
可恢复	默认 MemorySaver，可通过 <code>POSTGRES_URL</code> 切换到 PostgreSQL checkpoint
安全	API Key 通过 <code>.env</code> 注入，不写入代码仓库
错误处理	embedding 缓存损坏自动重置，Chroma metadata 序列化后再写入
可解释性	通过 <code>called_tools</code> 、 <code>iteration</code> 、 <code>missing_hint</code> 和 <code>final_docs</code> 记录 Agent 行为
可评估性	使用 Recall@K、MRR@K、nDCG@K 和索引一致性检查评价检索质量

SDD 约束的重点是把“模型能力”转化为“工程边界”。例如，工具调用可控意味着 Agent 不能随意执行未知动作，只能在 `vector_search` 和 `bm25_search` 等已注册工具中选择；状态可追踪意味着每轮检索产生的工具、结果、评估结论都保存在 `RetrievalState` 中，便于调试和复现；数据一致约束则确保 ChromaDB、BM25 语料和 parent_store 不会各自漂移。

三、系统架构与设计（15 分）

本章重点展示系统架构、Agent 交互流程和数据流设计，采用图文并茂方式说明核心模块如何协作。

3.1 系统总体架构

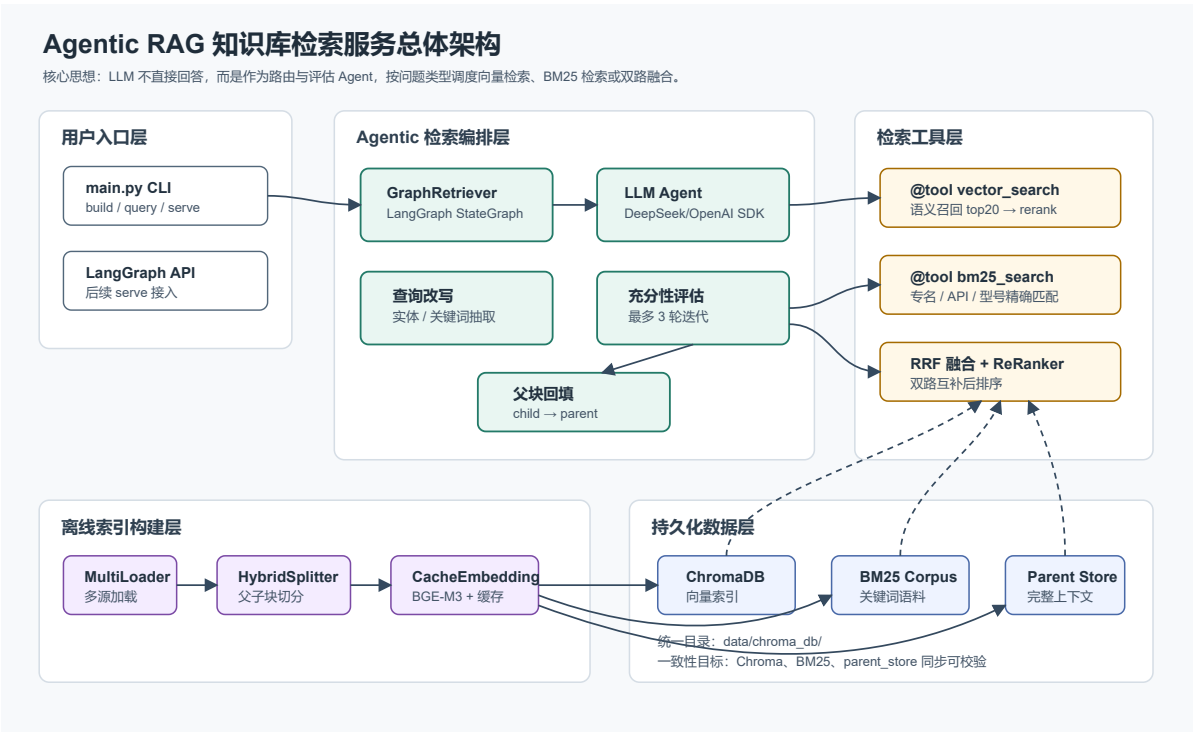
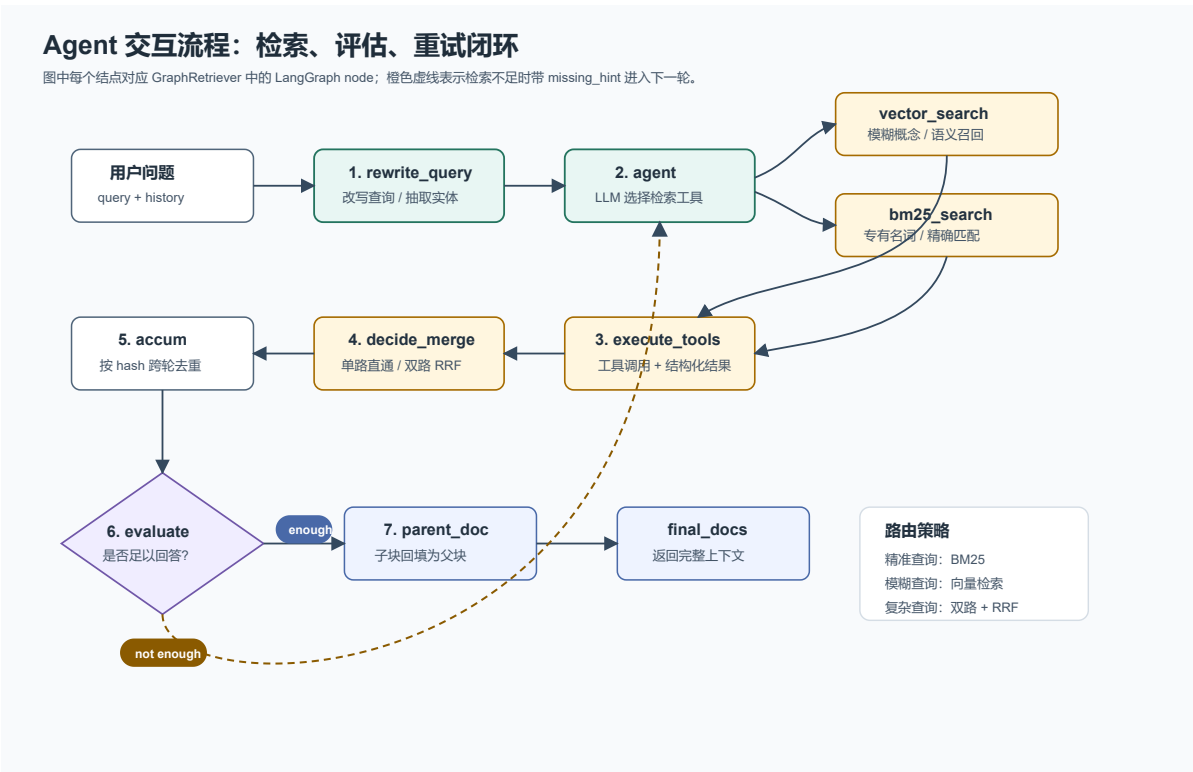


图1展示了项目的主要分层：用户入口层负责命令分发，Agentic 检索编排层负责查询改写、工具选择、充分性评估和父块回填，检索工具层封装向量/BM25/RRF/精排，离线索引层负责将原始数据转换为可查询索引，持久化层保证在线检索时可复用索引。

分层设计的好处是降低模块耦合。CLI 层只关心命令参数，不直接处理检索细节；GraphRetriever 只关心 Agent 状态流转，不直接读取数据集；检索器只提供稳定的 `invoke` 接口，不感知上层是否由 Agent 调用。后续如果要把 CLI 替换为 Web API，或把本地 ChromaDB 替换为远程向量库，只需要替换对应层的实现。

3.2 Agent 交互流程

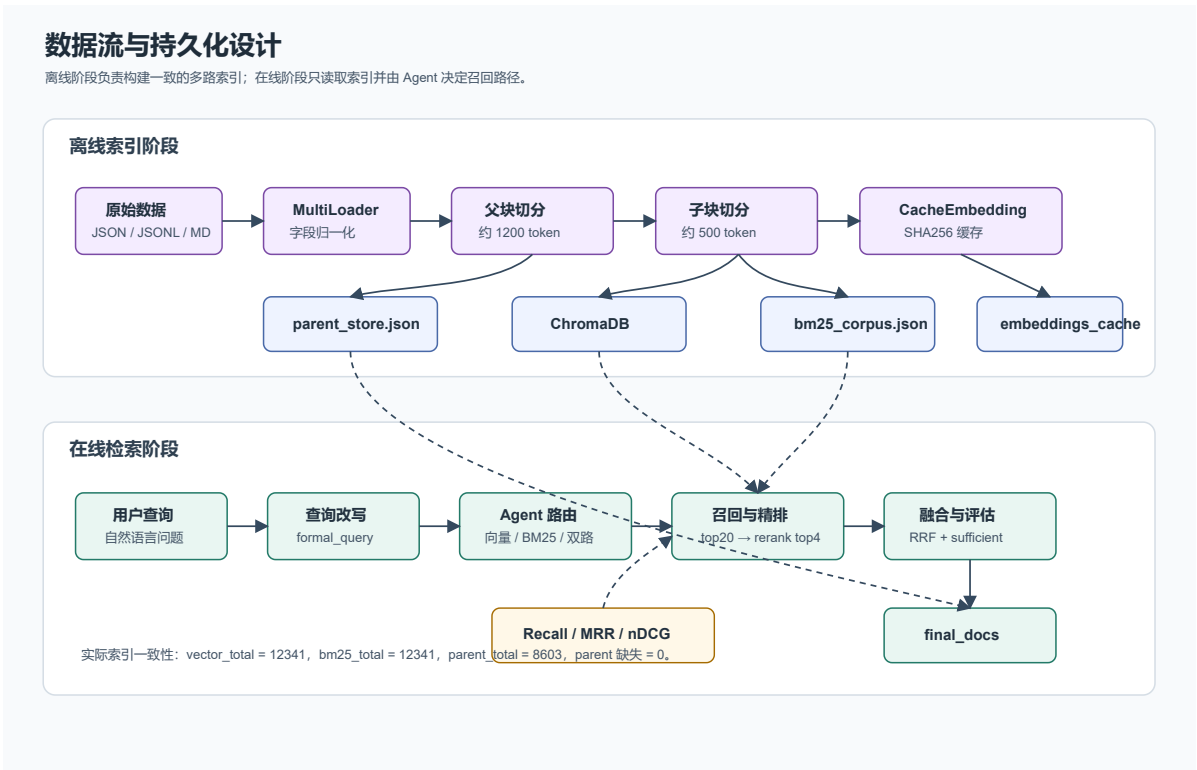


Agent 每轮先选择工具，再执行检索和融合，然后评估现有文档是否足以回答原始问题。当结果不足且未超过最大轮次时，系统把 `missing_hint` 加入下一轮提示，使 Agent 调整检索策略继续查询。

这个循环解决了普通 RAG 的一个典型问题：一次召回失败后系统没有自我修正机会。例如用户问“某技术方案在可观测性上的缺陷和改进”，第一轮可能只召回到概念解释，缺少“改进措施”部分；充分性评估节点会把缺口总结成 `missing_hint`，下一轮 Agent 可改用关键词检索或双路检索补充证据。

`max_iterations` 限制为 3，避免 Agent 无限检索导致延迟失控。

3.3 数据流设计



数据流分为离线和在线两部分。离线阶段将原始文档处理为父块、子块、向量索引、BM25 语料和 embedding 缓存；在线阶段读取这些持久化结果，由 Agent 决策检索路径，最终返回完整父块上下文。

数据一致性是该设计的关键。子块写入 ChromaDB 时会保存 `chunk_hash` 和 `parent_id`，BM25 语料保存同样的文本和 metadata，parent_store 则以 `parent_id` 保存完整父块内容。这样在线检索命中任意子块后，都能回到同一个父块上下文；测试脚本也可以通过 hash 和 parent_id 检查三份持久化数据是否同步。

四、关键实现与代码展示（15 分）

本章说明 Agent 架构、检索工具、融合算法、父子块召回和工程化实现细节。

4.1 Agent 核心循环

`GraphRetriever` 使用 `LangGraph StateGraph` 定义检索流程，核心节点包括 `rewrite_query`、`agent`、`execute_tools`、`decide_merge`、`accum`、`evaluate` 和 `parent_doc`。其中 `evaluate` 后的条件路由是 Agentic RAG 与普通 RAG 的主要区别：普通 RAG 一次召回后直接回答，本项目会先判断资料是否充分，不足时继续迭代。

选择 `LangGraph` 的原因是检索过程天然具有状态和条件分支。`rewrite_query` 负责把口语化问题改写成更适合检索的 formal query；`agent` 根据 prompt 和历史状态选择工具；`execute_tools` 同时产出 LLM 可读的 `ToolMessage` 和系统可处理的结构化文档；`accum` 负责跨轮去重，避免重复文档污染上下文；`evaluate` 负责把“是否足够回答”显式化。相比把所有逻辑写在一个函数里，`StateGraph` 更容易观

察每个节点的输入输出，也方便将来加入 tracing。

核心状态字段如下：

```
1 class RetrievalState(TypedDict):
2     query: str
3     formal_query: str
4     messages: list
5     vector_result: list[dict]
6     bm25_result: list[dict]
7     accumulated_docs: list[dict]
8     sufficient: bool
9     missing_hint: str
10    iteration: int
11    final_docs: list[dict]
```

4.2 工具定义

系统把检索器封装为 LangChain tool，供 LLM Agent 调用：

```
1 @tool
2 def vector_search(query: str) -> str:
3     docs = _vector_retriever.invoke(query)
4     docs = _reranker.rerank(query, docs, top_k=4)
5     return _format_docs(docs)
6
7 @tool
8 def bm25_search(query: str) -> str:
9     docs = _bm25_retriever.invoke(query, k=4)
10    return _format_docs(docs)
```

这使 Agent 看到的是明确的能力边界，而不是任意执行代码。`vector_search` 适合语义召回，`bm25_search` 适合专有名词、API 名称和型号检索。

工具封装还带来两个工程收益。第一，工具描述本身就是 Agent 的操作说明，模型能根据 docstring 判断适用场景；第二，工具输出被格式化为简短文本，避免把大量原始文档直接塞回 LLM 消息中造成上下文浪费。与此同时，系统在 `_execute_tools_node` 中还会调用底层 retriever 保存结构化结果，用于后续 RRF、去重和父块回填。

4.3 RRF 融合

当 Agent 同时调用向量检索和 BM25 检索时，系统使用 Reciprocal Rank Fusion 合并结果：

```
1 RRF_score(d) =  $\sum 1 / (k + \text{rank}_i(d))$ , k = 60
```

同一文档在两路召回中出现时分数累加，能提高“既语义相关又关键词命中”的文档排名。

RRF 的优势是不用强行比较不同检索器的原始分数。向量检索的相似度分数和 BM25 的词频相关分数量纲不同，直接加权需要额外归一化，且容易受数据集分布影响；RRF 只关心每个检索器内部排名，把排名靠前的文档稳定抬升。`k=60` 起到平滑作用，避免某一路的第一名过度支配结果，同时让两路都靠前的文档获得更高融合分。

4.4 父子块设计

索引阶段先生成约 1200 token 的父块，再切分约 500 token 的子块。这个设计是为了平衡 RAG 中两个相互冲突的问题：切块太小会提高定位精度，但上下文容易不完整；切块太大能保留上下文，但向量化时多个主题被压缩到同一个 embedding 中，细节会被稀释，导致相似度检索不够敏感。

如果只使用小块召回，用户问到一个跨段落问题时，系统可能只命中定义句或局部例子，缺少前置条件、结论和限制说明，最终回答会变得断章取义。如果只使用大块召回，embedding 需要同时表达多个子主题，某个函数名、参数名或错误码的局部信息会被整段语义平均掉，向量检索很难把它排到前面。因此本项目采用“子块负责召回、父块负责回答”的父子块策略。

具体流程是：`HybridTextSplitter` 先按标题、段落、句号等结构边界生成父块，并为每个父块写入 `parent_id`；随后再把父块切成更短的子块，子块继承同一个 `parent_id` 并写入 ChromaDB 和 BM25 语料。在线检索阶段，Agent 命中的是更精细的子块；进入 `parent_doc` 节点后，系统按 `parent_id` 从 `parent_store` 中取回完整父块，作为最终 `final_docs` 返回。

这种做法兼顾了召回精度和回答完整性。子块让向量空间里的语义点更集中，能更容易命中具体事实；父块让最终上下文保留定义、解释、约束和相邻证据，减少 LLM 在回答阶段凭空补全。对于企业文档中的 API 说明、故障排查步骤和设计规范，这种策略尤其重要，因为这些内容常常需要同时引用局部参数和周边条件。

4.5 工程化实现

工程点	实现
增量构建	根据 <code>chunk_hash</code> 过滤已入库文档
批量写入	每 1000 个文档写入一批，避免大批量写入失败
缓存复用	<code>CacheEmbedding</code> 使用文本 SHA256 缓存 embedding
Metadata 兼容	list/dict metadata 写入 Chroma 前序列化为 JSON 字符串
环境隔离	LLM、embedding、reranker 配置均通过 <code>.env</code> 和 <code>config.yaml</code> 管理
AI IDE 使用	使用 Trae CN 辅助梳理 Agent 路由、测试清单和文档结构，代码通过 Git 管理

这些工程化设计主要服务于稳定性和可维护性。增量构建避免重复写入同一批文档，降低长期维护知识库时的索引膨胀风险；批量写入让大规模数据入库时即使中途失败，也能定位失败批次；embedding 缓存减少重复模型推理，适合课程项目中反复调试切分策略的场景；metadata 序列化则是为了适配 ChromaDB 对元数据类型的限制，避免 list/dict 直接写入导致运行时错误。

4.6 AI IDE 使用截图



图4展示了本项目使用 Trae CN / Codex 辅助开发的过程：围绕 Agentic RAG 报告结构、系统图、测试结果和评分点覆盖进行迭代。AI IDE 的作用是辅助设计梳理、代码检查和文档生成，最终实现仍以本仓库代码和自动化测试结果为准。

4.7 Git 提交质量说明

本项目将源码、测试、文档和配图按目录组织，核心实现位于 `src/`，测试位于 `tests/`，报告和架构图位于 `docs/`。`.gitignore` 排除了 `.env`、运行时数据、缓存和虚拟环境，避免提交密钥或大体临临时文件；同时显式允许 `docs/final_report.md` 与 `docs/assets/*.svg` 纳入版本管理，保证最终报告可追踪。

五、测试与评估（10 分）

本章从功能正确性、索引一致性、排序指标和演示流程四个角度验证系统效果。

5.1 测试策略

测试分为四类：索引写入完整性、RAG 排序指标公式、索引一致性汇总逻辑、真实落盘索引一致性检查。真实向量检索效果可通过 `--with-recall` 开启，指标包括 Recall@K、MRR@K 和 nDCG@K。

索引写入完整性测试关注“数据是否正确落盘”，包括 embedding 缓存、Chroma 分批写入、BM25 同步、parent_store 持久化和 metadata 规范化。指标公式测试关注“评估是否可信”，避免 Recall、MRR、nDCG 的公式实现出现回归。索引一致性测试关注“多份索引是否对齐”，检查 Chroma、BM25 和 parent_store 在数量、hash、parent_id 上是否一致。真实落盘索引检查则直接读取当前本地索引，验证项目不是只通过 mock 测试。

5.2 自动化测试命令

```
1 | python tests/run_all_tests.py
```

可选真实召回评估：

```
1 | EMBEDDING_DEVICE=cpu python tests/run_all_tests.py --with-recall --sample-size 10 --ks 1,5,10,20
```

5.3 当前测试结果

测试项	结果
索引写入完整性单元测试	8 个用例通过
RAG 排序指标公式单元测试	3 个用例通过
索引一致性汇总逻辑单元测试	2 个用例通过
真实落盘索引一致性检查	通过
综合测试	total: 4, failed: 0

图5 自动化测试结果截图

命令: python tests/run_all_tests.py。结果显示 4 个测试步骤全部通过，真实落盘索引一致性 ok: True。

```
rag_service terminal

$ python tests/run_all_tests.py

== 索引写入完整性单元测试 ==
.....
Ran 8 tests in 0.224s
OK
PASS: 索引写入完整性单元测试

== RAG 排序指标公式单元测试 ==
...
Ran 3 tests in 0.000s
OK
PASS: RAG 排序指标公式单元测试

== Summary ==
total: 4, failed: 0
note: jieba/pkg_resources warning appeared, but all tests exited with code 0.

== 索引一致性汇总逻辑单元测试 ==
..
Ran 2 tests in 0.000s
OK
PASS: 索引一致性汇总逻辑单元测试

== 真实落盘索引一致性检查 ==
vector_total: 12341
bm25_total: 12341
parent_total: 8603
2wikimqa_e: vector=6866, bm25=6866
passage_retrieval_en: vector=5475, bm25=5475
vector_missing_parent_store: 0
ok: True
PASS: 真实落盘索引一致性检查
```

图5为基于真实命令输出整理的测试结果截图式证据图。原始命令为 `python tests/run_all_tests.py`，实际输出摘要如下：

```
1 | == 索引写入完整性单元测试 ==
2 | Ran 8 tests in 0.210s
3 | OK
4 | PASS: 索引写入完整性单元测试
5 |
6 | == RAG 排序指标公式单元测试 ==
7 | Ran 3 tests in 0.000s
8 | OK
9 | PASS: RAG 排序指标公式单元测试
10 |
11 | == 索引一致性汇总逻辑单元测试 ==
12 | Ran 2 tests in 0.000s
13 | OK
```

```
14 PASS: 索引一致性汇总逻辑单元测试
15
16 == 真实落盘索引一致性检查 ==
17 vector_total: 12341
18 bm25_total: 12341
19 parent_total: 8603
20 vector_missing_parent_store: 0
21 ok: True
22
23 == Summary ==
24 total: 4, failed: 0
```

真实落盘索引一致性结果：

指标	数值
Chroma 向量子块总数	12341
BM25 语料子块总数	12341
parent_store 条目数	8603
2wikimqa_e 子块数	6866
passage_retrieval_en 子块数	5475
vector_missing_parent_store	0
一致性状态	ok: True

5.4 评估指标

指标	含义
Recall@K	标准文档是否出现在 top K 检索结果中
MRR@K	第一个正确文档的倒数排名，越靠前越高
nDCG@K	对排名靠后的命中进行对数折损，衡量排序质量
索引一致性	检查 Chroma、BM25、parent_store 是否数量和 hash 对齐

Recall@K 衡量“有没有找到正确文档”，适合判断召回能力；MRR@K 衡量“正确文档第一次出现得有多靠前”，适合评价排序对用户体验的影响；nDCG@K 通过对低排名结果做对数折损，进一步强调高位结果质量。三者结合比单独看命中率更稳妥，因为 RAG 最终传给 LLM 的上下文窗口有限，正确文档如果排得太靠后，实际回答阶段也可能无法使用。

5.5 Demo 流程

```
1 cd src
2 python main.py build --sample 100 --mode offline
3 python main.py query "对比微服务和单体架构在可观测性上的差异" --topk 5
```

预期现象：复杂问题触发双路检索，系统先召回子块，再根据评估结果进入父块回填，最终输出更完整的上下文文档。

六、系统升级与扩展（10 分）

本章从架构扩展、服务化、协议化和生产级治理几个方向规划后续演进。

6.1 可扩展架构

当前系统已经把 Agent 编排、工具定义、索引构建、检索器、评估脚本拆分为相对独立模块。后续可以在不重写主流程的情况下扩展更多工具，例如 Web 搜索、SQL 查询、代码库搜索或企业内部接口调用。

可扩展性的关键在于工具接口稳定。只要新工具遵循 LangChain tool 的输入输出形式，并在 GraphRetriever 初始化时注册，Agent 就可以在同一套状态图中选择它。例如加入 SQL 工具后，结构化数据问题可以走数据库查询；加入 Web 搜索工具后，时效性问题可以走外部检索；加入 MCP Server 后，其他 IDE 或 Agent 也可以把本项目作为知识库能力调用。

6.2 下一阶段计划

优先级	计划	价值
P0	补齐 langgraph.json 和模块级 graph 导出	使 python main.py serve 成为可用 API 服务
P1	增加 MCP Server 封装	让其他 Agent 或 IDE 直接调用检索能力
P1	增加结构化日志和 tracing	观察每轮工具选择、召回数量和充分性判断
P2	增加 Web UI / Demo 页面	便于课堂展示 Agent 决策过程
P2	接入 PostgreSQL checkpoint	支持跨进程会话恢复和历史检索
P3	云服务器部署	提供公网或校园网可访问 URL

6.3 生产级增强方向

方向	设计
错误处理	工具调用失败时记录工具名、查询词和异常原因，并允许 Agent 切换检索路径
安全防护	API Key 仅使用环境变量；后续 API 服务需加入鉴权和限流
可观测性	输出每轮 called_tools、文档数量、RRF 排名、evaluate 判断
性能优化	批量 embedding、向量库增量构建、reranker 延迟加载
评测体系	固定测试集、离线 benchmark、人工标注对比

6.4 加分项覆盖说明

加分项	分值	本项目覆盖情况
融合 MCP 协议 / Agentic RAG 等前沿技术	+3 分	当前已实现 Agentic RAG；下一阶段可将 <code>GraphRetriever.invoke()</code> 封装为 MCP Server 工具
云服务器部署并提供可访问 URL	+3 分	当前为本地 MVP；已规划 LangGraph API 服务化和云部署路径
课堂分享展示	+2 分	报告提供图1至图5，可直接用于课堂展示
达到生产级水平	+3 分	已覆盖错误处理、安全防护、可观测性、索引一致性测试，后续补 API 鉴权和 tracing

七、课程总结（10 分）

本章总结个人收获、工程思维变化和后续改进方向。

通过本项目，我对企业级应用软件设计有了三个更具体的认识。第一，AI 应用不能只停留在“调用大模型”，更重要的是把模型放进可控的工程流程中，让它承担合适的职责。第二，RAG 系统的效果不仅取决于 embedding 模型，也取决于数据切分、索引一致性、检索路径选择和结果评估。第三，企业级项目必须把测试、配置、安全和可扩展性作为设计的一部分，而不是最后补充的内容。

本项目仍有不足：`serve` 入口还需要补齐 LangGraph 服务配置，真实召回 benchmark 还可以扩大样本规模，Agent 工具调用过程也可以增加更细粒度的日志。后续我会优先完善 API 服务化和 MCP 封装，使该项目从课程 MVP 进一步演进为可被其他 Agent 调用的企业知识库检索组件。

提交内容自查

内容模块	对应章节	完成情况
选题与设计思想	第一章	已说明问题价值、创新性、现有方案不足、技术路线合理性
Specs 规格设计	第二章	已给出 Product Spec、Architecture Spec、API Spec、SDD 规格
系统架构与设计	第三章	已包含系统架构图、Agent 交互流程图、数据流设计图
关键实现与代码	第四章	已展示 Agent 核心循环、工具定义、RRF、父子块、配置与 AI IDE 使用截图
测试与评估	第五章	已记录自动化测试、真实索引一致性、评估指标和测试结果截图
升级扩展设想	第六章	已规划 API 服务、MCP 协议、云部署、可观测性、生产级增强
课程总结	第七章	已总结个人收获、工程思维转变、不足和课程建议

内容模块	对应章节	完成情况
加分项	第六章	已具备 Agentic RAG 设计，并明确 MCP、云部署、课堂展示、生产级路径